

Capitolo 3

Metodologia

Questo capitolo definisce direttamente il modello di questo lavoro e la procedura con cui viene addestrato. La Sezione 3.1 formalizza il problema come Dec-POMDP, istanziando stato, azione, transizione, fattore di sconto, ricompensa e topologia di rete. La Sezione 3.2 presenta l'architettura alla base del modello, MetaSTGAT (Wang et al., 2022), chiudendo con i meccanismi spaziali alternativi confrontati nel Capitolo 4. La Sezione 3.3 descrive la procedura di addestramento adottata. La Sezione 3.4 raccoglie in forma tabellare ogni differenza tra questo lavoro e il riferimento originale, definendo le configurazioni MetaSTGAT Pro e quella che ne replica fedelmente la procedura. Il capitolo si chiude con l'ambiente di simulazione costruito su CITYFLOW (Sezione 3.5).

3.1 Formalizzazione del problema

La Sezione 2.2 introduce l'MDP come la tupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ e la sua estensione al caso multi-agente, il Dec-POMDP a osservabilità locale: ogni intersezione i osserva solo il proprio stato, ma le sue azioni influenzano, con un ritardo proporzionale alla distanza spaziale, lo stato delle intersezioni adiacenti. Questa sezione istanzia ciascuna componente di quella tupla per una rete di N intersezioni, secondo la formulazione di problema della Sezione 2.5. Ad ogni istante t , l'intersezione i osserva uno stato $s_i^t \in \mathcal{S}$, sceglie un'azione $a_i^t \in \mathcal{A}$ e riceve una ricompensa r_i^t , mentre l'informazione viene propagata secondo la topologia del grafo stradale. Rispetto alla formulazione del riferimento originale, cambiano stato, azione e ricompensa, discussi nel dettaglio in Sezione 3.4; transizione, fattore di sconto e topologia di rete restano identici.

Stato ($\mathcal{S}$). Ogni intersezione ha 4 strade in ingresso, una per direzione cardinale, con 3 corsie per senso di marcia, per un totale di 12 corsie in ingresso e altrettante in uscita. A questa configurazione si aggiungono 8 fasi semaforiche selezionabili.

Lo stato è la concatenazione di tre componenti:

$$s_i^t = [n_i^t \parallel w_i^t \parallel p_i^t] \in \mathbb{R}^{d_0} \quad (3.1)$$

dove $n_i^t \in [0, 1]^{12}$ è il numero di veicoli per corsia in ingresso entro il raggio di visibilità d_{vis} (normalizzato dividendo per 30 e poi limitato tra 0 e 1), $w_i^t \in [0, 1]^{12}$ è il vettore dei tempi di attesa dei veicoli da più tempo in corsia ($\tanh(\frac{\text{attesa}}{300})$) e $p_i^t \in \{0, 1\}^8$ è la fase corrente in codifica one-hot, per un totale $d_0 = 32$. Il riferimento originale adotta invece uno stato composto solo da n_i^t e p_i^t (dimensione totale $d_0 = 20$), la stessa dimensione usata anche dalla configurazione **paper** di questo lavoro (§3.3), che ne riproduce fedelmente la procedura.

Il campo visivo ristretto e la componente w_i^t sono due modifiche di questo lavoro. Il campo visivo limitato nasce da due ragioni distinte: una pratica e una teorica. La prima mira a ridurre il divario tra simulazione e realtà, per il quale è essenziale limitare l'uso della strumentazione tecnica necessaria per la raccolta dei dati a un'area il più circoscritta possibile. La seconda trova fondamento nella cinematica dei veicoli: ogni veicolo in questa simulazione raggiunge una velocità di percorrenza massima $v_{\text{max}} = 11.1$ m/s (circa 40 km/h) dettata dal limite di velocità imposto su ciascuna strada. Ciò implica che in un intervallo di tempo fissato $\Delta t = \Delta t_{\text{green}} + \Delta t_{\text{yellow}} = 13$ s, un veicolo possa percorrere una distanza non superiore a

$$d_{\text{vis}} = v_{\text{max}} \cdot \Delta t \approx 144.3 \text{ m.}$$

Il calcolo esclude i secondi di "tutto-rosso", in cui il deflusso è interrotto. Il campo visivo è stato quindi ristretto anche per forzare l'agente RL a concentrarsi esclusivamente sui veicoli che una fase semaforica può effettivamente influenzare. Sebbene in questa architettura il cutoff spaziale sia assunto omogeneo per l'intera rete, il codice che lo implementa tiene nativamente in considerazione eventuali casi di limiti di velocità o intervalli temporali variabili; la riduzione del campo visivo, inoltre, non viene applicata se la strada risulta più breve del cutoff stesso. Questa limitazione percettiva è riservata ai soli modelli di reinforcement learning di questo studio e non viene applicata alla baseline reattiva (Max-Pressure).

La componente w_i^t ha la stessa motivazione del termine di ricompensa MaxRedWait_i^t discusso più sotto: dare al modello un segnale diretto e locale sull'attesa, necessario perché possa correlare le proprie decisioni con l'equità tra le direzioni e non solo con

il numero di veicoli in coda.

Da un punto di vista pratico tutte queste feature possono essere calcolate tramite sensori stradali posti ai punti di stop dell'incrocio e a distanze d_{vis} da esso.

Azione ($\mathcal{A}$). $a_i^t \in \{0, \dots, 7\}$ seleziona una delle otto fasi che verranno descritte in §3.5. La transizione di "tutto-rosso" è gestita internamente dal simulatore e non appartiene allo spazio delle azioni. Il riferimento originale lascia all'agente la scelta libera tra tutte le 8 fasi a ogni passo. Questo lavoro introduce invece un vincolo *anti-starvation* (che contrasta cioè l'eccessiva attesa dei veicoli all'incrocio), che invalida la fase corrente come azione se è già stata selezionata due volte consecutive, forzando la rotazione:

$$\mathcal{A}_i^t = \{0, \dots, 7\} \setminus \{a_i^{t-1}\} \quad \text{se } a_i^{t-1} = a_i^{t-2}, \quad \mathcal{A}_i^t = \{0, \dots, 7\} \text{ altrimenti} \quad (3.2)$$

Il numero due non è arbitrario: nelle primissime fasi di training, quando la politica è ancora vicina al caso e non ha alcuna preferenza appresa, un agente senza questo vincolo tende a bloccarsi sulla prima fase che produce un ritorno anche solo leggermente positivo, ripetendola indefinitamente; vietare la ripetizione oltre la seconda volta consecutiva rompe questo ciclo senza impedire a una fase di restare attiva per un tempo ragionevole quando è davvero la scelta migliore per due passi di fila. In termini di tempo simulato, il vincolo limita a due gli step consecutivi sulla stessa fase, cioè 30 s, di cui 26 di verde (il resto è "tutto-rosso", §3.5). La maschera resta attiva anche in fase di valutazione, non solo durante l'addestramento.

Transizione (P). Essendo l'algoritmo di addestramento (Double DQN) *model-free*, le probabilità di transizione $P(s' \mid s, a)$ non vengono modellate esplicitamente, ma campionate empiricamente. L'ambiente CITYFLOW è per natura deterministico: fissati lo stato completo del simulatore e l'azione congiunta di tutte le intersezioni, la transizione dell'intero sistema collassa su un singolo stato successivo con probabilità 1. Questo non vale però per la coppia locale (s_i^t, a_i^t) di una singola intersezione: il suo stato successivo dipende anche dalle azioni scelte dagli altri agenti e dai veicoli fuori dal campo visivo di i , coerentemente con l'osservabilità parziale del Dec-POMDP (§2.2).

Fattore di sconto (γ). In questo lavoro viene fissato $\gamma = 0.85$ per ogni modello, senza alcuna ottimizzazione iperparametrica dedicata.

Ricompensa (R). Il riferimento originale impiega esclusivamente una ricompensa di pura pressione:

$$r_i^t = -\frac{P_i^t}{100}, \quad P_i^t = \text{Incoming}_i^t - \text{Outgoing}_i^t \quad (3.3)$$

dove P_i^t è la *pressione dell'intersezione* i al tempo t , ovvero la differenza tra il numero di veicoli in entrata (Incoming_i^t) e in uscita (Outgoing_i^t) dall'intersezione al tempo t . Il modello di questo lavoro sostituisce questa ricompensa con:

$$r_i^t = \text{clip} \left(\frac{-(\text{Incoming}_i^t - \text{Passed}_i^t) - \alpha \cdot \text{MaxRedWait}_i^t - \text{Penalty}_i^t}{100}, -2, 0 \right) \quad (3.4)$$

dove Incoming_i^t è il numero di veicoli entrati nella finestra visiva dell'intersezione al tempo t , Passed_i^t il numero di veicoli che hanno superato l'incrocio tra t e $t+1$, MaxRedWait_i^t il tempo di attesa massimo, in secondi, tra tutti i veicoli che si trovano nelle corsie con segnale rosso in ingresso all'istante t , $\text{Penalty}_i^t = 50$ se $\text{Passed}_i^t = 0 \wedge \text{Incoming}_i^t > 0$ e 0 altrimenti, e $\alpha \geq 0$ l'iperparametro con cui si modula la penalizzazione per lo *starvation* (l'eccessiva attesa dei veicoli); questo lavoro ne confronta tre valori, $\alpha \in \{0.08, 0.04, 0.00\}$ (Capitolo 4). Il fattore di scala $1/100$ è introdotto per stabilizzare i gradienti. Per il calcolo di tutte queste metriche ciascun modello fa riferimento, se impostata, alla distanza d_{vis} .

Il termine Penalty_i^t scatta quando la fase scelta non fa passare nessun veicolo pur essendocene in coda, il caso del "verde a vuoto": senza una penalizzazione esplicita l'agente può restare a lungo, nelle prime fasi di training, su una scelta degenerare di questo tipo senza ricevere un segnale di correzione chiaro. Il valore 50 (0.5 dopo la scala $1/100$) è scelto per essere comparabile in scala al termine $(\text{Passed}_i^t - \text{Incoming}_i^t)$, non derivato analiticamente. Il termine $\alpha \cdot \text{MaxRedWait}_i^t$ regola invece il concetto cardine di questa tesi: l'iperparametro α modula quanto la ricompensa penalizzi l'attesa massima registrata a un'intersezione, permettendo di studiare il compromesso tra la riduzione del tempo di percorrenza medio e il mantenimento dell'equità tra le direzioni di marcia (Capitolo 4). Infine il *clip* tra -2 e 0 , con il limite inferiore calibrato sperimentalmente nel corso della sperimentazione, evita che i picchi occasionali dei termini precedenti (in particolare MaxRedWait_i^t , potenzialmente illimitato) destabilizzino l'apprendimento con aggiornamenti di gradiente sproporzionati.

Topologia di Rete (G). Le N intersezioni sono rappresentate dai nodi di un grafo $G = (V, E)$. Un arco orientato $(i, j) \in E$ esiste se un segmento stradale connette fisicamente i a j . A ogni nodo è inoltre aggiunto un arco verso se stesso (*self-loop*),

cosicché i layer di *graph attention* possano pesare tanto lo stato del nodo stesso quanto quello dei vicini. Sotto il profilo della generalizzazione spaziale, la presente architettura impone che lo stato $s_i^{(t)}$ e le *meta feature* dipendano unicamente da dati locali, senza alcun accesso all'identità del nodo i all'interno della rete. Il modello non conosce quindi l'identità né la posizione assoluta del nodo, ma ne conosce il grado: tra le meta-feature (§3.2) compare il numero di incroci adiacenti, che distingue un incrocio d'angolo, di bordo o interno, senza però rivelare dove si trovi quell'incrocio nella rete.

3.2 Architettura MetaSTGAT

Il modello di questo lavoro riprende l'architettura di MetaSTGAT (Wang et al., 2022). La scelta di questo modello come punto di partenza è dettata dalla volontà di avere un sistema in grado di coordinare in modo efficace gli incroci di una rete stradale, così da minimizzare i tempi di percorrenza medi dei veicoli e, tramite le modifiche discusse in §3.4, indagare il compromesso con l'equità tra i vari flussi di veicoli. A differenza di Fixed-Time, MetaSTGAT è un modello sensibile allo stato della rete per quanto riguarda la scelta delle fasi; a differenza di Max-Pressure, la ricompensa di questo lavoro (Equazione 3.4) introduce esplicitamente un compromesso tra *throughput* (la capacità di smaltimento dei veicoli) e *starvation* (l'eccessivo tempo di attesa dei veicoli), a costo di una richiesta di informazioni più dettagliata da raccogliere a ogni istante decisionale (§3.1) e di una fase di addestramento.

L'architettura si articola nei seguenti componenti, organizzati in una pipeline che elabora lo stato di ogni intersezione (s_i^t) portandolo ad una rappresentazione compatta, utilizzata per produrre i Q-value (ovvero una stima della ricompensa attesa per ciascuna azione):

- un encoder di stato duale
- una cella ricorrente (LSTM)
- due istanze di un layer di *graph attention* (GAT)
- due moduli di meta-learning, spaziale e temporale
- una layer lineare per calcolare i Q-value.

Questa sezione presenta ciascun componente, soffermandosi sulla ragione che ha condotto Wang et al. (2022) alla sua introduzione. Prima di introdurre la generalizzazione "meta", ci si concentra sulla presentazione dell'architettura STGAT, ovvero

la versione standard del modello priva del meccanismo di meta-apprendimento. La Figura 3.1 anticipa in forma schematica la pipeline appena elencata, utile come riferimento visivo durante la lettura dei paragrafi che seguono.

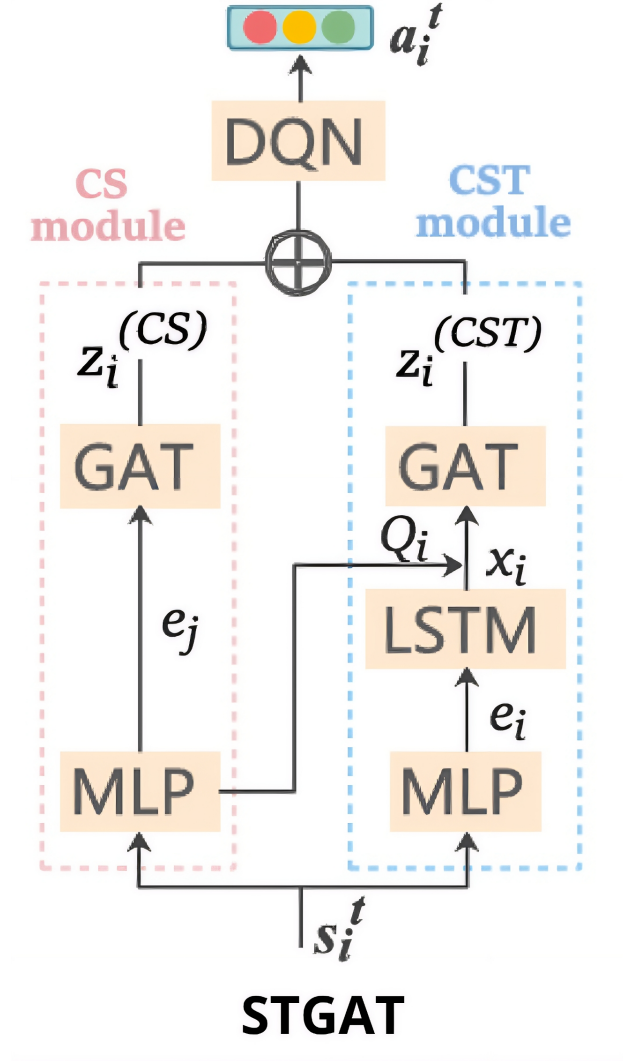


Figura 3.1: Schema a blocchi dell'architettura STGAT: dallo stato dell'intersezione s_i^t , attraverso l'encoder duale (MLP) e i moduli CS e CST (GAT e LSTM a pesi condivisi), fino all'azione a_i^t scelta dal Double DQN (il blocco "DQN" della figura, adattata dal riferimento originale, ne indica la testa dei Q-value). Adattato da Wang et al. (2022).

3.2.1 State Encoder duale

Lo stato $s_i^t \in \mathbb{R}^{d_0}$ ($d_0 = 32$ o 20 , §3.1) di un incrocio i al tempo t attraversa due Multilayer Perceptron (MLP) a due strati con la stessa architettura ma pesi e

inizializzazioni indipendenti:

$$e_i^t = \text{MLP}_t(s_i^t) = \sigma(W_{21} \sigma(W_{11}s_i^t + b_{11}) + b_{21}) \in \mathbb{R}^{d_1} \quad (3.5)$$

$$e_j^t = \text{MLP}_s(s_i^t) = \sigma(W_{22} \sigma(W_{12}s_i^t + b_{12}) + b_{22}) \in \mathbb{R}^{d_1} \quad (3.6)$$

dove d_1 viene fissato a 64, $W_{11}, W_{12} \in \mathbb{R}^{d_1 \times d_0}$ e $W_{21}, W_{22} \in \mathbb{R}^{d_1 \times d_1}$ sono i pesi, $b_{11}, b_{21}, b_{12}, b_{22} \in \mathbb{R}^{d_1}$ i bias e σ la funzione di attivazione ReLU. Il pedice j in e_j^t è un'etichetta del ramo spaziale, coerente con la notazione delle figure di questa sezione, non l'indice di un nodo vicino: anche e_j^t è calcolato a partire dallo stato s_i^t del nodo stesso.

Il vettore $e_i^t = \text{MLP}_t(s_i^t)$ verrà usato come input nella LSTM e veicolerà le caratteristiche temporali dello stato, mentre $e_j^t = \text{MLP}_s(s_i^t)$ verrà usato come input nelle GAT, catturando le caratteristiche spaziali dello stesso.

3.2.2 Modulo spazio-temporale corrente (CST)

Nel momento di una decisione, un conteggio istantaneo dei veicoli, da solo, non distingue una coda che si sta formando da una che si sta esaurendo. Due situazioni che possono richiedere una fase diversa appaiono dunque identiche a un modello privo di memoria. Il primo dei due moduli di STGAT viene introdotto per questo motivo: combina l'informazione temporale accumulata da un'intersezione con l'informazione spaziale corrente di sé stessa e dei suoi vicini, così che la scelta dell'azione possa tenere conto di come il traffico più ravvicinato si sta evolvendo, non solo di come si presenta localmente nell'istante attuale. Wang et al. (2022) motivano questo modulo osservando che altri lavori sulle Spatio-Temporal Graph Neural Networks (STGNN) propongono strutture simili, ma vi combinano le informazioni spaziali e temporali in un unico embedding oppure le usano separatamente; con il modulo CST cercano invece di intrecciare le due informazioni in modo più elaborato.

Il ramo temporale dell'encoder e_i^t (§3.2.1) attraversa anzitutto una cella LSTM con pesi condivisi da ogni nodo della rete:

$$\begin{bmatrix} f_i^t \\ \iota_i^t \\ o_i^t \\ g_i^t \end{bmatrix} = \begin{bmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{bmatrix} \left(W_{\{f,\iota,o,g\}}^\top [e_i^t \parallel h_i^{t-1}] + b_{\{f,\iota,o,g\}} \right), \quad (3.7)$$

$$c_i^t = f_i^t \odot c_i^{t-1} + \iota_i^t \odot g_i^t,$$

$$x_i^t = h_i^t = o_i^t \odot \tanh(c_i^t);$$

in notazione compatta:

$$(h_i^t, c_i^t) = \text{LSTM}(e_i^t; h_i^{t-1}, c_i^{t-1}). \quad (3.8)$$

Il vettore $x_i^t := h_i^t$ prodotto da questa cella rappresenta concettualmente un riassunto temporale dei flussi di traffico locali.

Lo stesso vettore, nel meccanismo di attenzione che segue, diventa l'informazione che i vicini dell'incrocio i "leggono" per decidere quanto pesare il contributo di i nel proprio contesto locale. Questa trasmissione di informazione avviene tramite l'applicazione di una Graph Attention Network (GAT) con una rappresentazione differente rispetto a quella della GAT standard di [Veličković et al. \(2018\)](#) riportata nel Capitolo 2. In tale GAT infatti ciascun nodo viene comparato con gli altri nodi secondo una stessa proiezione dell'embedding. Nella struttura proposta invece, prendendo ispirazione da approcci di machine translation ([Vaswani et al., 2017](#)), i nodi si influenzano vicendevolmente secondo valori distinti: Query, Key e Value. In particolare si scelgono come Query $Q = e_j^t$ (la codifica spaziale corrente del nodo ricevente) e come Key e Value $K = V = x_i^t$ (la memoria temporale di ciascun vicino). L'idea alla base di questa scelta è il tentativo di arricchire le caratteristiche temporali correnti di ciascun nodo con la memoria temporale dei flussi dello stesso e dei nodi vicini. Si precisa che nel caso di $H > 1$ teste di attenzione, come in questo lavoro, i valori Q, K, V non sono proiettati da matrici distinte per ciascuna testa, bensì il vettore a d_1 componenti viene partizionato in H segmenti contigui di dimensione $d_h = d_1/H$, e la testa h opera sul solo blocco $Q_h(i) := Q(i)[hd_h : (h+1)d_h]$ (analogamente per $K_h(u), V_h(u)$). Le teste restano quindi distinte perché leggono porzioni diverse dello stesso embedding, non perché dispongano di proiezioni apprese proprie.

Formalmente, nel caso in esame, per ciascun nodo i con vicinato $\mathcal{N}(i)$ (con $i \in \mathcal{N}(i)$) e h una delle $H = 4$ teste di dimensione $d_h = d_1/H = 16$, questo modulo di GAT viene descritto come segue:

$$\varphi_h(i, u) = \frac{Q_h(i) \cdot K_h(u)}{\sqrt{d_h}}, \quad (3.9)$$

$$\alpha_h(i, u) = \frac{\exp(\varphi_h(i, u))}{\sum_{u' \in \mathcal{N}(i)} \exp(\varphi_h(i, u'))},$$

$$z_h(i) = \sum_{u \in \mathcal{N}(i)} \alpha_h(i, u) V_h(u). \quad (3.10)$$

L'output finale di questo modulo spazio-temporale corrente sarà

$$z_{\text{CST}}(i) = [z_1(i) \parallel \dots \parallel z_H(i)] \in \mathbb{R}^{d_1}. \quad (3.11)$$

Auspiciabilmente, in questo stato è racchiusa una sintesi delle dinamiche spaziali e temporali del traffico locale e degli incroci adiacenti.

3.2.3 Modulo spaziale corrente (CS)

Il modulo spazio-temporale, filtrato dalla memoria della cella ricorrente, può attenuare segnali che sono invece rilevanti nell'istante presente; ad esempio un picco di traffico simultaneo su un intero asse necessita di coordinamento spaziale puro, a prescindere dalla durata del picco stesso. Questo secondo modulo, strutturalmente simile al primo, elabora perciò la sola informazione spaziale corrente e_j^t , senza alcuna componente di memoria. Il vettore e_j^t attraversa una GAT che opera secondo le stesse dinamiche descritte nel paragrafo precedente, ma con l'assegnazione $Q = K = V = e_j^t$. L'output finale del modulo spaziale corrente sarà

$$z_{\text{CS}}(i) = [z_1(i) \parallel \dots \parallel z_H(i)] \in \mathbb{R}^{d_1}, \quad (3.12)$$

Tale rappresentazione del traffico permetterà al modello di focalizzarsi sulla coordinazione dei nodi rispetto alla sola fotografia presente della rete.

3.2.4 Testa Q-value

Le due uscite, $z_{\text{CST}}(i)$ e $z_{\text{CS}}(i)$, catturano quindi aspetti complementari, l'uno informato dalla memoria temporale dei vicini, l'altro dalla sola situazione presente. Tali vettori vengono infine concatenati e attraversano un singolo layer lineare per produrre la stima del valore di ciascuna delle 8 azioni eseguibili all'incrocio i e all'istante t :

$$q(s_i^t, \cdot) = W_q [z_{\text{CST}}(i) \parallel z_{\text{CS}}(i)] + b_q \in \mathbb{R}^8 \quad (3.13)$$

dove $W_q \in \mathbb{R}^{8 \times 2d_1}$ e $b_q \in \mathbb{R}^8$.

3.2.5 Meta-knowledge learner

I due moduli appena descritti, alla base della struttura STGAT, condividono gli stessi pesi per ogni intersezione della rete, indipendentemente dal ruolo che occupa. Ad esempio un incrocio periferico, uno pienamente interno alla rete stradale, o uno attraversato da un'arteria a densità di traffico elevata (§3.5.3) applicano esattamente

la stessa regola di coordinamento. MetaSTGAT introduce un meccanismo di meta-apprendimento che condiziona tale regola su un embedding di caratteristiche locali dell'incrocio, cosicché i layer di GAT e di LSTM possano specializzarsi rispetto alle condizioni osservate da ciascun nodo invece di applicare la stessa funzione ovunque. L'idea alla base di questo meccanismo, condivisa da entrambi i componenti "meta" che seguono, è illustrata in Figura 3.2: le meta-feature spaziali e temporali, elaborate da un meta-learner dedicato, generano i pesi dei layer di LSTM e GAT al posto di parametri fissi condivisi da ogni nodo.

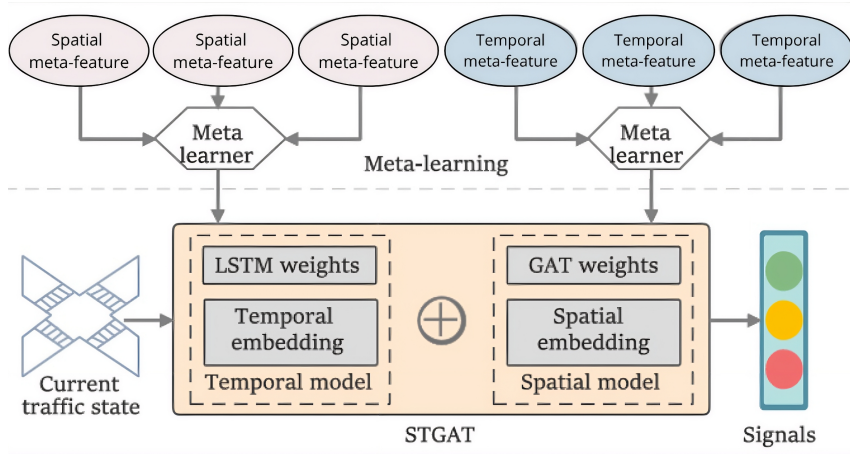


Figura 3.2: Principio del meta-apprendimento applicato da MetaSTGAT: le meta-feature spaziali e temporali condizionano, tramite un meta-learner dedicato, i pesi dei layer LSTM e GAT usati per elaborare lo stato corrente del traffico. Adattato da Wang et al. (2022).

In particolare, due MLP a due strati, uno per l'aspetto spaziale e uno per quello temporale del problema, apprendono un embedding di caratteristiche locali dell'intersezione (Spatial Meta-Knowledge, SMK; Temporal Meta-Knowledge, TMK) a partire dalle meta-feature che descrivono il nodo (x_i^{smk} e x_i^{tmk}):

$$\text{SMK}(i) = \text{MLP}_{\text{smk}}(x_i^{\text{smk}}) \in \mathbb{R}^{64} \quad (3.14)$$

$$\text{TMK}(i) = \text{MLP}_{\text{tmk}}(x_i^{\text{tmk}}) \in \mathbb{R}^{64} \quad (3.15)$$

Come già accennato, tali valori verranno poi usati per generare i pesi di Meta-GAT e Meta-LSTM, tramite dei MLP addizionali specializzati per ciascuno dei moduli, secondo quanto riporteremo in (§3.2.6) e (§3.2.7). Il passaggio dalle meta-feature grezze ai due embedding $\text{SMK}(i)$ e $\text{TMK}(i)$ è riassunto graficamente in Figura 3.3.

Le meta-feature x_i^{smk} e x_i^{tmk} riprendono l'insieme di informazioni del riferimento originale, adattato in questo lavoro; sono pensate per fornire al modello informazio-

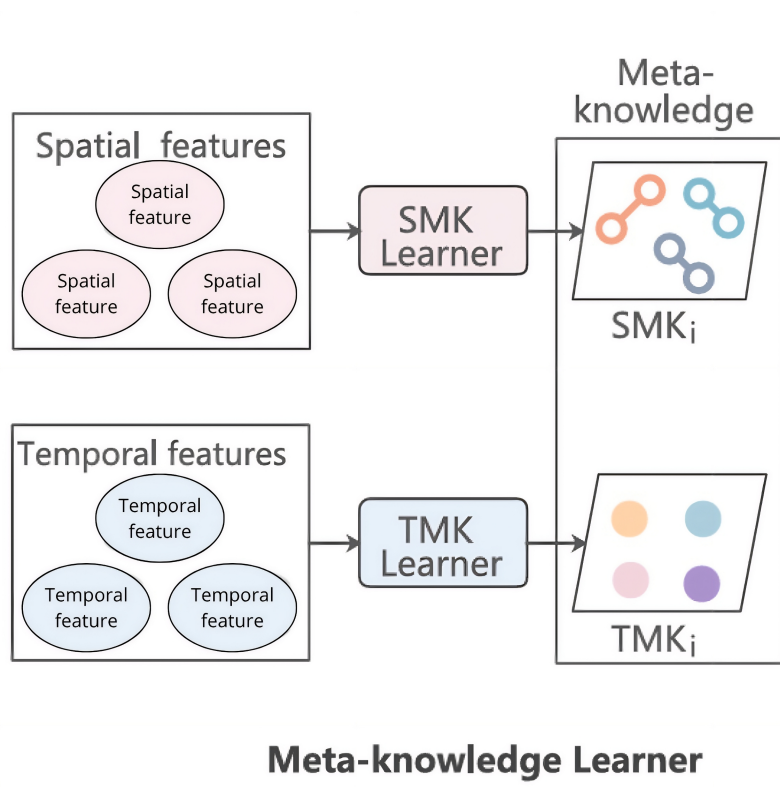


Figura 3.3: Il Meta-knowledge Learner: le meta-feature spaziali e temporali grezze vengono elaborate rispettivamente dallo SMK-Learner e dal TMK-Learner, producendo gli embedding $SMK(i)$ e $TMK(i)$ usati da Meta-GAT e Meta-LSTM. Adattato da Wang et al. (2022).

ni aggiuntive che ne consentano una migliore specializzazione rispetto al contesto specifico di ciascun incrocio.

Il vettore spaziale $x_i^{\text{smk}} \in \mathbb{R}^{26}$ concatena cinque gruppi di feature, ciascuno variabile da nodo a nodo:

- pressione per corsia [12]: per ciascuna corsia, la differenza tra i veicoli in ingresso sulla corsia e la media dei veicoli sulle corsie in uscita raggiungibili da essa, divisa per 30;
- numero di incroci adiacenti [1]: frazione tra il numero di vicini del nodo e il massimo previsto dalla topologia a griglia (4), distingue quindi un incrocio d'angolo, di bordo o interno;
- differenza tra la pressione media propria e quella di ciascun incrocio adiacente [4]: per ciascuno dei vicini, la pressione media delle 12 corsie del nodo meno quella del vicino, con zero-padding negli slot oltre il numero di vicini effettivamente presenti;
- asimmetria di pressione tra le corsie N/S e le corsie E/O dello stesso nodo [1]: media della pressione sulle 6 corsie N/S meno la media sulle 6 corsie E/O;
- pressione per fase [8]: la stessa quantità dell'Equazione 2.12, usata da Max-Pressure (§2.5.2) per decidere la fase da selezionare, una componente per ciascuna delle 8 fasi.

Il costo di raccolta di queste feature si limita a sensori con la stessa capacità visiva già richiesta per lo stato principale e per il reward, sia sulle corsie in ingresso sia su quelle in uscita: le feature di pressione si misurano semplicemente contando i veicoli su queste corsie, mentre il numero di incroci adiacenti è un dato topologico costante per ciascun incrocio.

Il vettore di meta-feature temporali $x_i^{\text{tmk}} \in \mathbb{R}^{25}$ concatena invece tre diversi tipi di informazione, tutte variabili nel tempo all'interno dello stesso nodo:

- variazione della coda per corsia rispetto a 3 passi precedenti [12]: per ciascuna corsia, differenza tra il numero di veicoli in coda all'istante corrente e quello di 3 step prima;
- durata di permanenza della fase corrente [1]: numero di step consecutivi in cui la fase è rimasta attiva, saturato e normalizzato in $[0, 1]$ rispetto a un tetto di 2 step, oltre il quale la maschera anti-starvation di questo lavoro (§3.1) impone comunque un cambio di fase;

- deviazione standard della coda per corsia sulla finestra recente [12]: calcolata sugli ultimi 5 step disponibili.

Anche queste feature vengono calcolate a partire da informazioni fornite dal simulatore e non richiedono strumenti aggiuntivi.

Si noti come nessuna di queste feature entri anche nello stato principale: la scelta è legata alla volontà di non appesantire lo stato del sistema, sfruttando le stesse per agire direttamente sui pesi che regolano la trasmissione dell'informazione.

3.2.6 Meta-GAT

La versione meta delle GAT usate in STGAT riprende la struttura architetturale delle stesse, ma implementa delle modifiche mirate a consentire ai layer di apprendere regole di aggregazione spaziale dipendenti dalle caratteristiche del singolo nodo. I valori assegnati a Q, K, V rimangono invariati, sia per il modulo spaziale ($Q = K = V = e_j^t$) sia per il modulo spazio-temporale ($Q = e_j^t, K = V = x_i^t$). L'unica differenza riguarda l'inserimento, nel punteggio di compatibilità dell'Equazione 3.9, di una trasformazione lineare per nodo che in STGAT non esiste: il modulo CS riceve il proprio meta-embedding da $\text{SMK}(i)$, il modulo CST da $\text{TMK}(i)$, in modo che la regola di attenzione spaziale e quella spazio-temporale si adattino a caratteristiche locali diverse. In particolare, per ciascun nodo i e ciascuna testa h , una rete $\text{MLP}_{\text{GAT}, \text{smk}, h}$ condivisa da tutti i nodi genera, dal solo embedding meta del nodo ricevente, i pesi del modulo spaziale:

$$W_{j,h}^{(i)}, b_{j,h}^{(i)} = \text{MLP}_{\text{GAT}, \text{smk}, h}(\text{SMK}(i)) \in \mathbb{R}^{d_h \times d_h}, \mathbb{R} \quad (3.16)$$

e analogamente, tramite $\text{MLP}_{\text{GAT}, \text{tmk}, h}$, per il modulo spazio-temporale:

$$W_{k,h}^{(i)}, b_{k,h}^{(i)} = \text{MLP}_{\text{GAT}, \text{tmk}, h}(\text{TMK}(i)) \in \mathbb{R}^{d_h \times d_h}, \mathbb{R} \quad (3.17)$$

dove le iper-reti MLP sono composte da due layer, condivise da tutti i nodi della rete: solo il loro output dipende da i , tramite l'input $\text{SMK}(i)$ o $\text{TMK}(i)$. E dunque, per ogni nodo $u \in \mathcal{N}(i)$:

$$\varphi_h(i, u) = \frac{(W_h^{(i)} Q_h(i)) \cdot K_h(u) + b_h^{(i)}}{\sqrt{d_h}}, \quad (3.18)$$

$$\alpha_h(i, u) = \frac{\exp(\varphi_h(i, u))}{\sum_{u' \in \mathcal{N}(i)} \exp(\varphi_h(i, u'))},$$

$$z_h(i) = \sum_{u \in \mathcal{N}(i)} \alpha_h(i, u) V_h(u),$$

$$z(i) = [z_1(i) \parallel \dots \parallel z_H(i)] \in \mathbb{R}^{d_1},$$

con $W_h^{(i)}$ e $b_h^{(i)}$ relativo al proprio modulo di riferimento. La Figura 3.4 mette a confronto i due moduli appena descritti, evidenziando come SMK e TMK alimentino iper-reti distinte per generare i pesi (W, b) specifici di ciascuno.

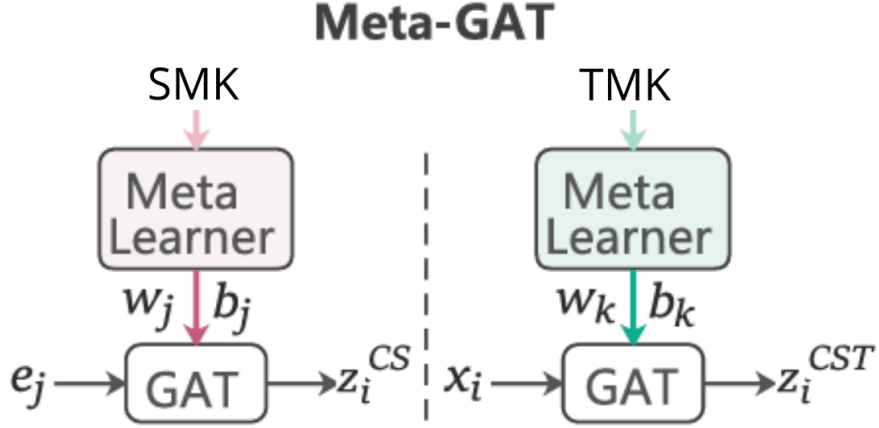


Figura 3.4: Meta-GAT: il modulo CS (sinistra) genera i pesi (W_j, b_j) della propria GAT a partire da SMK, il modulo CST (destra) genera (W_k, b_k) a partire da TMK; entrambi producono l'output z_i del rispettivo modulo. Adattato da Wang et al. (2022).

3.2.7 Meta-LSTM

Meta-LSTM applica alla cella ricorrente di STGAT la stessa generalizzazione che Meta-GAT applica al *graph attention*: i pesi $W_{(f,\ell,o,g)}$ e i bias $b_{(f,\ell,o,g)}$ condivisi in STGAT da ogni nodo della rete, sono qui sostituiti da pesi e bias generati individualmente per ogni nodo.

Un MLP a due layer per ciascun gate genera, dal solo $\text{TMK}(i)$, i pesi $W_{(f,\ell,o,g)}^{(i)} \in \mathbb{R}^{2d_1 \times d_1}$ e i bias $b_{(f,\ell,o,g)}^{(i)} \in \mathbb{R}^{d_1}$, modificando l'Equazione 3.7 come segue:

$$\begin{bmatrix} f_i^t \\ \ell_i^t \\ o_i^t \\ g_i^t \end{bmatrix} = \begin{bmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{bmatrix} \left(\left(W_{\{f,\ell,o,g\}}^{(i)} \right)^\top [e_i^t \parallel h_i^{t-1}] + b_{\{f,\ell,o,g\}}^{(i)} \right) \quad (3.19)$$

$$c_i^t = f_i^t \odot c_i^{t-1} + \ell_i^t \odot g_i^t \quad (3.20)$$

$$x_i^t = h_i^t = o_i^t \odot \tanh(c_i^t) \quad (3.21)$$

In notazione compatta, esplicitando la sola dipendenza aggiuntiva dai pesi generati dal meta-learner:

$$(h_i^t, c_i^t) = \text{LSTM}(e_i^t; h_i^{t-1}, c_i^{t-1} \mid W_{\{f,\ell,o,g\}}^{(i)}, b_{\{f,\ell,o,g\}}^{(i)}), \quad x_i^t := h_i^t. \quad (3.22)$$

L'output x_i^t alimenta il modulo CST di Meta-GAT come K e V (§3.2.6), analogamente a quanto visto con STGAT. L'architettura appena descritta è riassunta graficamente in Figura 3.5, dove W_Φ, b_Φ indicano genericamente i pesi e i bias generati dal meta-learner.

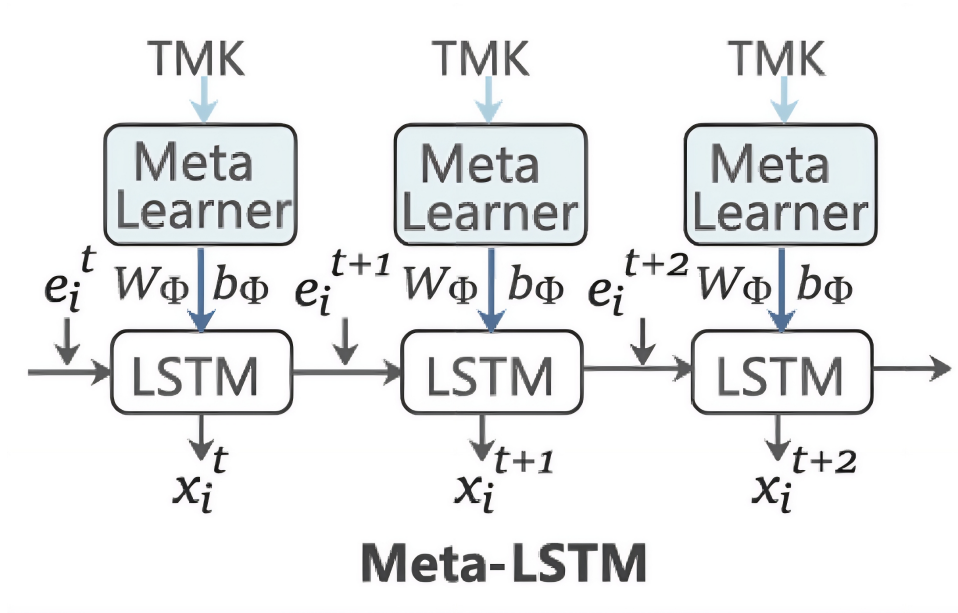


Figura 3.5: Meta-LSTM "srotolata" nel tempo: a ogni istante t il meta-learner genera, da $TMK(i)$, i pesi W_Φ e b_Φ della cella LSTM che produce x_i^t dall'input e_i^t e dallo stato nascosto precedente h_i^{t-1}, c_i^{t-1} . Adattato da Wang et al. (2022).

Messi insieme, i moduli descritti in questa sezione compongono l'architettura di Figura 3.6, che ne riassume il flusso complessivo dallo stato s_i^t all'azione scelta.

3.2.8 Meccanismi spaziali alternativi

Il modulo Meta-GAT decide quanto ciascun vicino debba contribuire all'aggiornamento di un'intersezione tramite un punteggio di compatibilità appreso, specifico per ogni nodo (§3.2.6). Questa formulazione rappresenta tuttavia solo una delle possibili declinazioni del paradigma di *message passing* su grafo. Con l'obiettivo di esplorare soluzioni alternative, in questa sezione vengono introdotte ed estese in chiave meta-learning due diverse architetture di aggregazione spaziale: Meta-GCN

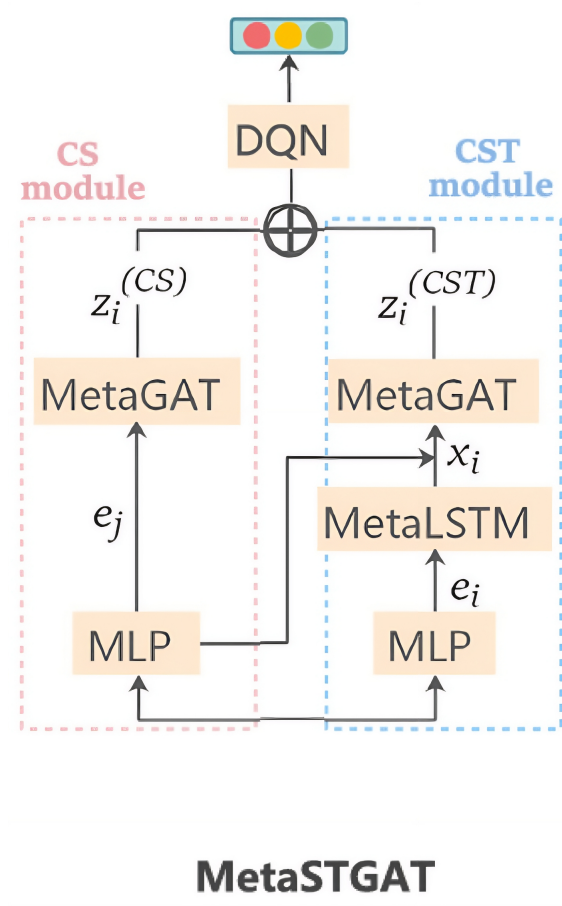


Figura 3.6: Schema a blocchi completo di MetaSTGAT: rispetto a STGAT (Figura 3.1), GAT e LSTM sono sostituiti dalle rispettive versioni meta-condizionate, con i pesi generati dagli embedding $SMK(i)$ e $TMK(i)$ del Meta-knowledge Learner. Adattato da Wang et al. (2022).

e Meta-SONAR, presentate come sostituzione del solo blocco GAT di Figura 3.6, a parità di tutto il resto.

Ogni altro componente dell'architettura (encoder di stato, meta-learner spaziale e temporale, Meta-LSTM, testa Q-value) e la procedura di training (§3.3) rimangono invariati: a parità di tutto il resto, le differenze di prestazione misurate nel Capitolo 4 si possono quindi ricondurre principalmente al solo meccanismo di aggregazione spaziale, entro la variabilità tra seed.

La prima architettura, una *graph convolution network* (GCN), rimuove il punteggio di compatibilità della GAT e lo sostituisce con un peso fisso determinato solo dalla topologia. La seconda, una propagazione ondulatoria a lungo raggio chiamata SONAR (Trenta et al., 2025), sostituisce l'aggregazione a un passo con l'evoluzione discretizzata di un sistema dinamico su più iterazioni interne.

In entrambi i casi il principio "meta" resta identico a Meta-GAT: i pesi dell'aggregazione sono generati da un embedding SMK/TMK per ciascun nodo, non appresi come parametro fisso condiviso.

3.2.8.1 Meta-GCN

La GCN (Kipf and Welling, 2017), introdotta nel Capitolo 2 (Equazione 2.7), pesa il contributo di ogni vicino con la normalizzazione simmetrica del grado: una quantità che dipende solo dalla topologia della rete, mai dal contenuto dello stato dei nodi coinvolti. È dunque il termine di paragone più diretto per isolare cosa aggiunge il punteggio di compatibilità di Meta-GAT: sostituendolo con questa regola fissa, e lasciando invariato tutto il resto dell'architettura, ogni differenza di prestazioni misurata nel Capitolo 4 è imputabile a quel solo meccanismo.

Per un nodo i , con $\mathcal{N}(i)$ il suo vicinato (incluso il self-loop permanente di questo lavoro, §3.1) e $\hat{d}_i = |\mathcal{N}(i)|$ il grado corrispondente, il principio meta genera W e b per ciascun nodo ricevente i , a partire dal suo embedding meta m_i (pari a SMK(i) per il modulo CS e a TMK(i) per il modulo CST), tramite un'iper-rete a due strati condivisa da tutti i nodi, in modo analogo a quanto visto in Meta-GAT:

$$W^{(i)}, b^{(i)} = \text{MLP}_{\text{GCN},m}(m_i) \in \mathbb{R}^{d_1 \times d_1}, \mathbb{R}$$

Poi, per ogni nodo i , la regola di aggregazione, che nella versione originale non-meta della GCN userebbe una matrice W condivisa da ogni nodo della rete (Equazione 2.7), diventa:

$$z(i) = \sigma \left(\left(\sum_{u \in \mathcal{N}(i)} \frac{1}{\sqrt{\hat{d}_i \hat{d}_u}} W^{(i)} V(u) \right) + b^{(i)} \right) \quad (3.23)$$

dove σ è la funzione di attivazione ReLU e $V(u)$ il valore del nodo u (h_u nell'Equazione 2.7 del Capitolo 2). Come per Meta-GAT, il modulo CST assegna $V = x_i^t$ (l'uscita della Meta-LSTM, §3.2.7) e $m_i = \text{TMK}(i)$; il modulo CS assegna $V = e_j^t$ e $m_i = \text{SMK}(i)$.

3.2.8.2 Meta-SONAR

Sia Meta-GAT sia Meta-GCN propagano informazione a un solo passo di distanza per ogni applicazione del layer: raggiungere un vicino a k archi di distanza richiede k applicazioni. SONAR (Trenta et al., 2025), introdotto nel Capitolo 2 (§2.4), evita questo costo modellando la propagazione come un sistema dinamico continuo, la cui evoluzione nel tempo, non nello spazio dei layer, porta l'informazione a più hop di distanza in un'unica applicazione. Nella formulazione originale, richiamata in dettaglio nel Capitolo 2, la conduttanza a_{ui} , la dissipazione $D(\cdot)$ e la forzante $F(\cdot)$ dell'Equazione 2.11 sono reti a pesi fissi, condivise da tutti i nodi, senza alcuna nozione di meta-apprendimento. In questo lavoro W nell'Equazione 2.10 è l'identità: il termine si riduce così al solo accoppiamento spaziale pesato dalla conduttanza a_{ui} , un peso scalare per coppia di nodi, analogo al punteggio di compatibilità di Meta-GAT o al peso di normalizzazione del grado di Meta-GCN.

Questo lavoro applica il principio meta alla sola conduttanza a_{ui} , l'analogo diretto del peso appreso per coppia di nodi di Meta-GAT e Meta-GCN: una rete a due strati genera, dall'embedding meta m_i del nodo ricevente i , i parametri $W_a^{(i)} \in \mathbb{R}^{d_1 \times 1}$ e $b_a^{(i)} \in \mathbb{R}$, da cui

$$a_{ui} = \text{ReLU}\left((X_u - X_i)W_a^{(i)} + b_a^{(i)}\right) \quad (3.24)$$

la ReLU garantisce $a_{ui} \geq 0$, coerente con l'interpretazione di a_{ui} come conduttanza. Dissipazione $D(\cdot)$ e forzante $F(\cdot)$ restano reti a pesi fissi applicate allo stato corrente del nodo (un solo layer lineare seguito da ReLU per $D(\cdot)$, due layer lineari con una ReLU intermedia per $F(\cdot)$): sono filtri locali, non meccanismi di comunicazione tra nodi, per cui il legame con le caratteristiche locali codificate da SMK/TMK è più debole che per la conduttanza.

La posizione iniziale X^0 e la velocità iniziale V^0 sono prodotte in modo analogo a Meta-GAT: per il modulo CST $X^0 = x_i^t$ (l'uscita della Meta-LSTM) e $V^0 = W_V e_j^t$ con $m_i = \text{TMK}(i)$, per il modulo CS: $X^0 = e_j^t$ e $V^0 = W_V e_j^t$ con $m_i = \text{SMK}(i)$, dove $W_V \in \mathbb{R}^{d_1 \times d_1}$ è una matrice di proiezione fissa e condivisa da tutti i nodi, analoga in questo a $D(\cdot)$ e $F(\cdot)$.

A differenza di Meta-GAT e Meta-GCN, dove la profondità spaziale si ottiene impilando più applicazioni del layer a pesi indipendenti, in Meta-SONAR le L iterazioni dell'Equazione 2.11 condividono gli stessi pesi $W_a^{(i)}$, la stessa proprietà di

costo costante rispetto a L già discussa per la formulazione originale nel Capitolo 2. Questo lavoro confronta $L = 2$ e $L = 4$ iterazioni, con passo di integrazione $h = 0.1$ in entrambi i casi (Capitolo 4).

3.3 Procedura di addestramento

I modelli di questo lavoro, a meno di ablazioni specifiche, sono addestrati con Double DQN su sequenze campionate da un replay buffer prioritizzato, con burn-in e Back-propagation Through Time (BPTT), Huber loss pesata dagli *importance-sampling weight* di PER, hard update della target network e gradient clipping; l'ottimizzatore è RMSprop per fedeltà al riferimento originale.

Algoritmo di Deep Reinforcement Learning. Il riferimento originale impiega DQN standard. Il modello di questo lavoro adotta invece Double DQN (van Hasselt et al., 2016) (§2.2), che disaccoppia selezione e valutazione dell'azione nel calcolo del target, riducendo il bias di sovrastima del valore Q tipico di DQN.

Funzione di loss. Le componenti di Double DQN, Huber loss e PER sono definite in astratto, per una singola transizione, in §2.2; questo paragrafo formalizza il modo in cui si combinano sulle sequenze ricorrenti e multi-nodo di questo lavoro.

Sia N il numero di intersezioni della rete e ℓ_{seq} la lunghezza di una sequenza campionata k , di cui le prime ℓ_{burn} istanze servono solo al burn-in per inizializzare lo stato ricorrente (h, c) , mentre le successive $\ell_{\text{grad}} = \ell_{\text{seq}} - \ell_{\text{burn}}$ istanze contribuiscono al gradiente. Per ciascun nodo i e ciascun passo t successivo al burn-in, il target Double DQN è:

$$y_i^t = r_i^t + \gamma Q(s_i^{t+1}, \underset{a'}{\operatorname{argmax}} Q(s_i^{t+1}, a'; \theta); \theta^-) \quad (3.25)$$

dove online e target network elaborano lo stesso stato ricorrente propagato dal burn-in. L'errore di differenza temporale è

$$\delta_i^t = Q(s_i^t, a_i^t; \theta) - y_i^t,$$

e la loss della sequenza k è la Huber loss media su tutti i nodi e i passi validi:

$$\mathcal{L}_k = \frac{1}{N \ell_{\text{grad}}} \sum_{i=1}^N \sum_t L_{\kappa}(\delta_i^t) \quad (3.26)$$

con soglia di Huber $\kappa = 1$ (il valore di libreria usato senza ritocchi, §2.2). Il campionamento dal replay buffer avviene per sequenze intere, non per transizioni isolate, di conseguenza anche la priorità del PER è definita a livello di sequenza, come la media di $|\delta_i^t|$ sugli stessi nodi e passi di $\mathcal{L}_k$, e aggiornata dopo ogni batch. La loss ottimizzata a ogni passo di gradiente è la media, sulle B sequenze del batch, delle loss $\mathcal{L}_k$ pesate dai rispettivi coefficienti di importance sampling w_k :

$$\mathcal{L} = \frac{1}{B} \sum_{k=1}^B w_k \mathcal{L}_k. \quad (3.27)$$

Il calcolo di $\mathcal{L}$ è seguito da un solo passo di RMSprop e dal gradient clipping descritti sotto.

Calcolo del gradiente. Il riferimento originale allena la componente ricorrente (LSTM) senza una fase di burn-in, calcolando il gradiente a partire dall’istanza campionata con lo stato ricorrente (h, c) non ricostruito, un disallineamento tra lo stato ricorrente osservato durante la raccolta dati e quello ricostruito in training. Qui si campionano invece sequenze di $\ell_{\text{seq}} = 4$ transizioni dal replay buffer, con un burn-in di $\ell_{\text{burn}} = 2$ passi secondo lo schema R2D2 (Kapturowski et al., 2019): i passi di burn-in aggiornano (h, c) senza contribuire al gradiente, correggendo il disallineamento appena descritto, mentre i successivi $\ell_{\text{grad}} = 2$ passi aggiornano i pesi del modello. Si applica infine un *gradient clipping*, limitando a norma 1 il gradiente, al fine di stabilizzare la convergenza.

Replay buffer. Il riferimento originale utilizza un replay buffer con una capacità di 10.000 transizioni congiunte (l’intero stato della rete a un singolo istante) e con campionamento uniforme. Si adotta qui invece un replay buffer organizzato per sequenze, con Prioritized Experience Replay (PER, Schaul et al., 2016), la relativa correzione tramite pesi di *importance sampling* (§2.2) e una capacità di 4.800 transizioni congiunte, equivalenti a 40 episodi di storia. Il PER concentra l’aggiornamento dei pesi sulle sequenze con errore di stima più alto, anziché trattare queste ultime allo stesso modo delle transizioni già ben predette; limitare la capacità del buffer agli ultimi episodi, invece di conservarne l’intera storia, mantiene l’esperienza di training coerente con una politica che ha già imparato qualcosa, invece di continuare a ricampionare le traiettorie quasi casuali dei primissimi episodi. La configurazione **paper** (§3.4), che riproduce fedelmente la procedura del riferimento originale, torna alla capacità di 10.000 transizioni e al campionamento uniforme.

Funzione di loss (confronto). Il riferimento originale impiega l'errore quadratico medio (MSE) tra Q predetto e target. Si predilige qui la Huber loss (§2.2), più robusta agli outlier della MSE, pesata dai coefficienti di importance sampling introdotti dal PER.

Ottimizzatore e target network. L'ottimizzatore RMSprop è usato con *learning rate* 10^{-3} , mantenendo i restanti iperparametri (fattore di smoothing, ε numerico, momentum) ai valori di libreria, non ritoccati in questo lavoro. L'aggiornamento della target network è una copia periodica integrale dei pesi della rete online, non una media mobile: ogni 100 step di ottimizzazione, il bersaglio dell'Equazione 3.25 viene sostituito per intero. Con 100 aggiornamenti del gradiente per episodio (paragrafo successivo) e una frequenza di sostituzione di 100 step, la target network viene quindi sincronizzata esattamente una volta per episodio.

Gli iperparametri usati nel presente studio sono riassunti in Tabella 3.1.

Ciclo di training. Lo pseudocodice seguente descrive il ciclo esterno di addestramento; le equazioni di questa sezione ne dettagliano il calcolo della loss.

Algorithm 1 Ciclo di addestramento (per modello)

```

1: Inizializza rete online  $\theta$ , rete target  $\theta^- \leftarrow \theta$ , replay buffer  $\mathcal{D}$ 
2: for 10 episodi di warm-up do
3:   Esegui un episodio con  $\varepsilon = 1$  (azioni interamente casuali), popola  $\mathcal{D}$ 
4: end for
5: for episodio = 1, ..., 100 do
6:   if episodio mod 10 = 0 then
7:     Esegui l'episodio con  $\varepsilon = 1$ , senza aggiornare  $\theta$  (esplorazione ciclica)
8:   else
9:     Esegui l'episodio con la politica  $\varepsilon$ -greedy corrente, popola  $\mathcal{D}$ 
10:    for 100 aggiornamenti del gradiente do
11:      Campiona un batch di  $B$  sequenze da  $\mathcal{D}$  (PER)
12:      Calcola  $\mathcal{L}$  (Equazioni 3.26–3.27), aggiorna le priorità
13:      Un passo di RMSprop su  $\theta$ , con gradient clipping
14:      if numero totale di aggiornamenti mod 100 = 0 then
15:         $\theta^- \leftarrow \theta$  (hard update della target network)
16:      end if
17:    end for
18:    Decadi  $\varepsilon$  verso il suo minimo (raggiunto al 60% degli episodi)
19:  end if
20: end for

```

L'idea alla base dell'esplorazione ciclica è quella di continuare a inserire nel buffer esperienza che la politica ormai quasi deterministica non visiterebbe più spon-

Parametro	Valore	Origine	Motivazione
learning rate	10^{-3}	questo lavoro	–
γ	0.85	riferimento originale	–
α (equità)	$\{0.08, 0.04, 0.00\}$	questo lavoro	sweep, Capitolo 4
d_1	64	questo lavoro	–
teste H	4	riferimento originale	–
d_{vis}	144.3 m	valore derivato	$v_{\text{max}}(\Delta t_{\text{green}} + \Delta t_{\text{yellow}})$
Penalty	50 (0.5 dopo /100)	questo lavoro	comparabile in scala a (Passed–Incoming)
clip ricompensa	$[-2, 0]$	questo lavoro	stabilizzare il gradiente da MaxRedWait
ε -greedy	$0.9 \rightarrow 0.01$	questo lavoro	decadimento $\approx 0.928/\text{episodio}$, minimo all’episodio 60
esplorazione ciclica	1 episodio ogni 10 a $\varepsilon = 1$	questo lavoro	–
batch size B	20	riferimento originale	–
capacità buffer	4.800 transizioni congiunte	questo lavoro	= 40 episodi
ℓ_{seq} , burn-in	4, 2	questo lavoro	schema R2D2
PER α	0.6	libreria	–
PER β	$0.4 \rightarrow 1.0$, lineare su 10^5 frame	libreria	–
PER ζ	10^{-5}	libreria	evita priorità nulla
Huber κ	1.0	libreria	–
gradient clipping (norma)	1.0	questo lavoro	–
ottimizzatore	RMSprop, solo lr impostato	libreria (resto)	fedeltà al riferimento originale
target update	hard, ogni 100 step	questo lavoro	1 sync/episodio
dimensioni nascoste	64 ovunque	questo lavoro	encoder, SMK/TMK, iperreti
SONAR L, h	$\{2, 4\}, 0.1$	questo lavoro	–
episodi, warm-up	100, 10	questo lavoro	warm-up interamente casuale
update per episodio	100	riferimento originale	–
seed di training	1 per modello	questo lavoro	nessuno sweep multi-seed in training
artery boost	3.0	questo lavoro	pesa l’instradamento, non aggiunge veicoli
software	Python 3.10, PyTorch 2.13 (CPU), torch_geometric 2.8, CityFlow 0.1	questo lavoro	–

Tabella 3.1: Iperparametri di addestramento e architettura del presente studio.

taneamente, al fine di dare al modello la possibilità di non convergere su soluzioni sub-ottimali.

3.4 Differenze rispetto al riferimento originale

La Tabella 3.2 raccoglie in forma sintetica ogni differenza tra questo lavoro e il riferimento originale, introdotta e argomentata nel dettaglio nelle sezioni precedenti.

Componente	Wang et al. (2022)	Questo lavoro	Sezione
Campo visivo	illimitato	limitato a $d_{\text{vis}} \approx 144.3$ m	§3.1
Stato	n_i^t, p_i^t ($d_0 = 20$)	aggiunge w_i^t ($d_0 = 32$)	§3.1
Ricompensa	pressione pura (Eq. 3.3)	Passed–Incoming, Max-RedWait, Penalty, clip (Eq. 3.4)	§3.1
Maschera azioni	nessuna	anti-starvation, 2 fasi consecutive	§3.1
Meta-feature	definite dagli autori	adattate (es. tetto a 2 step legato alla maschera)	§3.2
Algoritmo	DQN	Double DQN	§3.3
Ricorrenza	nessun burn-in	burn-in 2 + BPTT 2 (R2D2)	§3.3
Replay buffer	10.000 transizioni, uniforme	4.800 transizioni, PER	§3.3
Loss	MSE	Huber, pesata da PER	§3.3
Gradient clipping	assente	norma 1	§3.3
Svolta a destra	libera, senza attendere il verde	consentita solo se autorizzata dalla fase	§3.5

Tabella 3.2: Sintesi delle differenze tra questo lavoro e il riferimento originale.

A partire da queste differenze, questo lavoro definisce due configurazioni di riferimento, usate nel Capitolo 4: **MetaSTGAT Pro** applica tutte le modifiche elencate in Tabella 3.2, ed è la configurazione a cui questo capitolo fa riferimento salvo indicazione contraria; **paper** riproduce invece fedelmente la procedura di Wang et al. (2022), tornando ai valori della colonna centrale della tabella. Il Capitolo 4 valuta inoltre, tramite uno studio di ablazione dedicato, il contributo di ciascun gruppo di modifiche isolatamente, tramite configurazioni intermedie tra queste due.

3.5 Ambiente di simulazione

L'ambiente di simulazione si basa su CITYFLOW (Zhang et al., 2019), un simulatore microscopico open source con motore in C++ e binding in Python, progettato specificamente per il controllo semaforico su larga scala tramite reinforcement learning, per le ragioni già discusse nella Sezione 2.5.3. La Figura 3.7 ne mostra il visualizzatore su una singola intersezione.

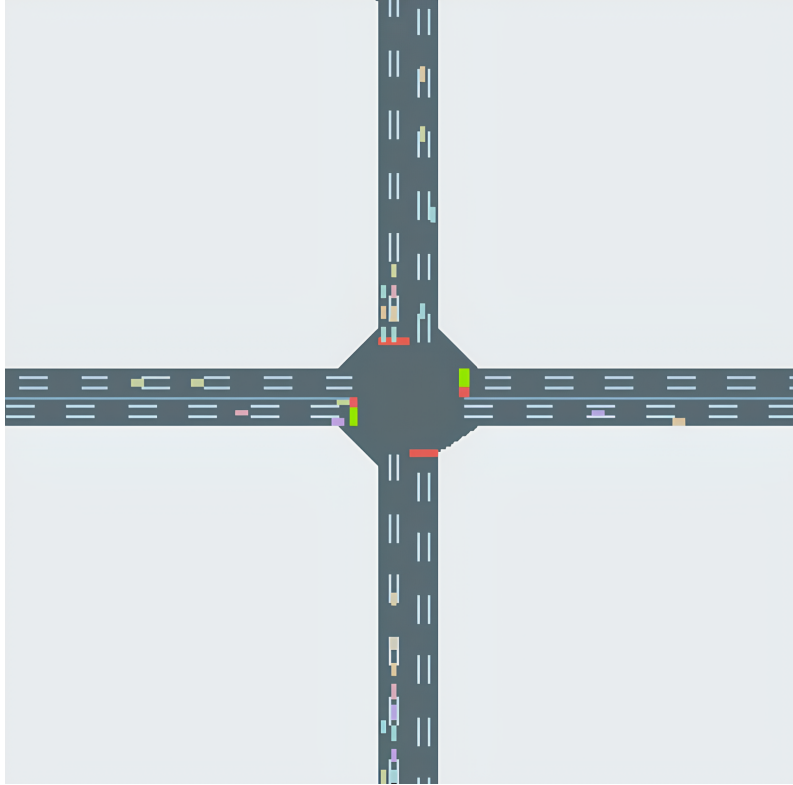


Figura 3.7: Rendering del visualizzatore di CITYFLOW per una singola intersezione a quattro vie: le indicazioni verde/rosso ai margini dell'incrocio mostrano i movimenti abilitati dalla fase corrente, i rettangoli colorati sulle corsie i veicoli in transito o in coda.

3.5.1 Struttura della rete e mappatura delle fasi

La topologia della rete è codificata in un file di configurazione (*roadnet*), che definisce l'insieme delle intersezioni (nodi) e dei segmenti stradali (archi) con le relative molteplicità di corsia. Per ogni intersezione il *roadnet* elenca le fasi semaforiche disponibili, ciascuna definita come l'insieme dei movimenti (coppie corsia di origine e di destinazione) abilitati in quella fase. L'azione dell'agente si riduce pertanto alla selezione dell'indice corrispondente alla fase da attivare. Ogni intersezione di questo lavoro modella 12 corsie in ingresso (quattro direzioni cardinali, tre corsie

ciascuna: sinistra, diritto, destra) e 8 fasi semaforiche selezionabili. La Tabella 3.3 riporta la mappatura di tali fasi (Figura 3.8), adottando la nomenclatura N/S/E/W (*North/South/East/West*) e T/L (*Through*, diritto, e *Left*, sinistra).

Fase	Nome	Movimenti abilitati
0	NTST	Nord diritto+destra, Sud diritto+destra
1	NLSL	Nord sinistra, Sud sinistra
2	NTNL	Nord sinistra+diritto+destra
3	STSL	Sud sinistra+diritto+destra
4	WTET	Ovest diritto+destra, Est diritto+destra
5	WLEL	Ovest sinistra, Est sinistra
6	ETEL	Est sinistra+diritto+destra
7	WTWL	Ovest sinistra+diritto+destra

Tabella 3.3: Le 8 fasi semaforiche e i movimenti che ciascuna abilita.

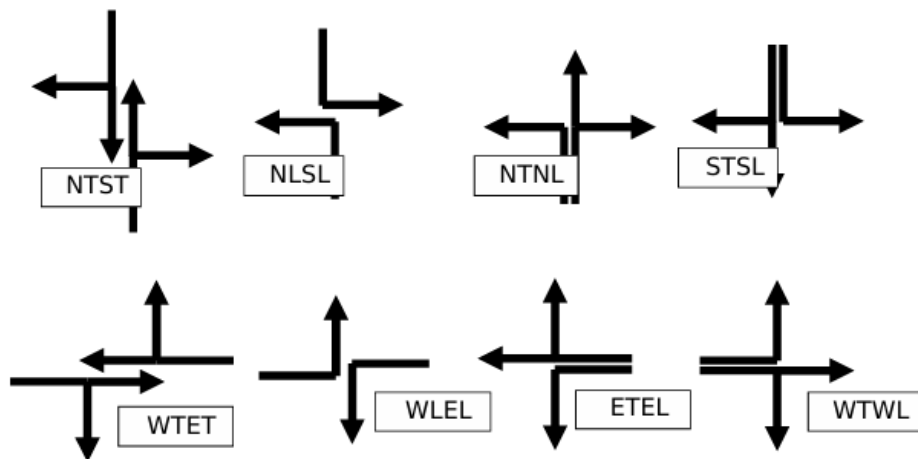


Figura 3.8: Rappresentazione grafica delle 8 fasi di Tabella 3.3: le frecce indicano i movimenti abilitati da ciascuna fase.

Il riferimento originale permette ai veicoli che svoltano a destra di muoversi liberamente, senza attendere il semaforo verde, a patto che non vi siano veicoli provenienti da altre direzioni. Al fine di prevenire una qualunque possibile interferenza tra diversi flussi e garantire un controllo più rigoroso, si è deciso di non adottare questo comportamento, permettendo la svolta a destra solo quando esplicitamente autorizzata dalla fase semaforica. L'insieme delle fasi sopra riportate rappresenta una configurazione tale da sfruttare al massimo lo spazio delle direzioni possibili, evitando al contempo conflitti tra le traiettorie dei veicoli.

3.5.2 Gestione dell'interverde

L'interverde è il periodo di tempo che intercorre tra la fine del verde di una fase semaforica e l'inizio del verde di quella successiva. Il suo scopo è quello di garantire la sicurezza stradale, permettendo ai veicoli che si trovano nell'intersezione di sgombrarla prima che venga attivata la fase successiva. Il motore di simulazione CITYFLOW non modella nativamente tale fase transitoria, nello specifico, non gestisce i semafori gialli e il conseguente passaggio sicuro tra due fasi consecutive; dunque l'interverde deve essere costruito esplicitamente. In questo lavoro, l'intervallo decisionale dell'agente (lo *step* di simulazione) ha una durata complessiva $\Delta t_{\text{step}} = 15$ s simulati, idealmente ripartita in tre sotto-fasi, con le stesse durate del riferimento originale: verde ($\Delta t_{\text{green}} = 10$ s), giallo ($\Delta t_{\text{yellow}} = 3$ s) e "tutto-rosso" ($\Delta t_{\text{red}} = 2$ s). Per il limite di CITYFLOW sopra descritto e in questo studio accettato come compromesso, l'ambiente assimila il giallo al verde senza imporre modelli di decelerazione specifici, mantenendo pertanto i movimenti autorizzati dalla policy per una durata totale di $\Delta t_{\text{green}} + \Delta t_{\text{yellow}} = 13$ s consecutivi. Al termine di questa finestra temporale, viene eseguita per $\Delta t_{\text{red}} = 2$ s una fase che non è presente tra le azioni selezionabili dal modello, ovvero il semaforo rosso in contemporanea su tutte le corsie. Questo intervallo di tutto-rosso viene eseguito a ogni step, indipendentemente dal fatto che la fase successiva coincida o meno con quella corrente: anche selezionare due volte di fila la stessa fase comporta i 2 s di tutto-rosso. Questo intervallo di guardia è necessario per garantire lo svuotamento dell'intersezione prima del cambio di fase, così da prevenire collisioni. A questo punto lo step giunge al termine e il modello formula una nuova decisione, scegliendo quale delle 8 fasi implementare nel ciclo successivo.

È importante sottolineare che questa accortezza riduce di fatto il tempo effettivo utile di decisione del singolo agente di controllo semaforico, in quanto due dei quindici secondi di ogni step sono dedicati alla fase "tutto-rosso" e quindi da considerarsi tempo morto (rappresentando circa il 13% del tempo totale).

3.5.3 Dataset e generazione parametrica

Ogni roadnet e ogni flusso di traffico usato in questo lavoro è generato parametricamente e in modo interamente deterministico da uno script dedicato, senza alcun campionamento empirico. Questo approccio consente di avere un pieno controllo sulle variabili operative come la struttura della rete, la densità veicolare e presenza di arterie principali, oltre a garantire l'esatta riproducibilità degli scenari di simula-

zione, un requisito metodologico fondamentale per un equo confronto tra modelli di controllo semaforico.

Topologie e spaziature. L'ambiente di simulazione implementa tre topologie a griglia regolare di dimensioni 4×4 (16 intersezioni), 5×5 (25 intersezioni) e 6×6 (36 intersezioni), impiegate per l'addestramento e per valutare la capacità di generalizzazione dei modelli rispetto alla scala spaziale.

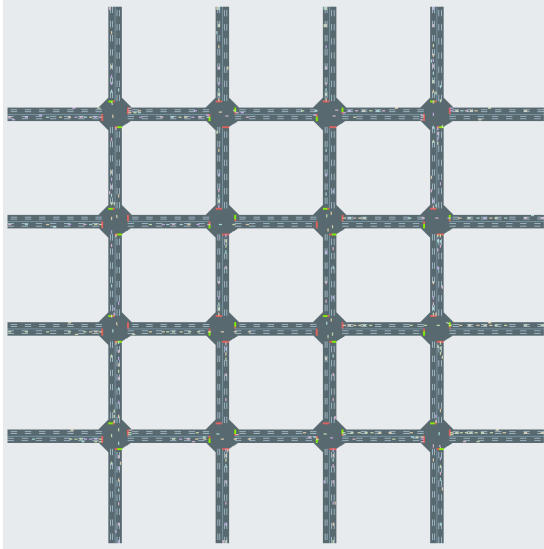
Un aspetto rilevante nella scelta della spaziatura tra incroci è la comparsa spontanea della cosiddetta onda verde, uno standard strutturale nell'ingegneria del traffico contemporanea: un flusso di veicoli che si propaga attraverso la rete senza fermarsi ai semafori. La spaziatura tra incroci è pertanto di due tipi: calibrata per l'onda verde e non calibrata. In assenza di offset temporali, in quanto le transizioni di fase sono sincrone per tutti i nodi della rete, l'unico modo per ottenere un'onda verde è calibrare la distanza fisica tra gli incroci sulla velocità di marcia e sulla durata del ciclo: alla velocità di riferimento $v_{\max} = 11.1$ m/s, in un ciclo di $\Delta t_{\text{step}} = 15$ s viene percorsa una distanza pari a 166.5 m. Dunque la prima spaziatura proposta è proprio di questa misura, ma ne faremo riferimento con l'etichetta "100m". La spaziatura non calibrata invece è 100m più estesa e viene indicata con "200m".

Le roadnet non condividono tutte lo stesso ruolo. La griglia 4×4 con spaziatura "100m" compare sia in training sia in test, mentre le altre tre combinazioni topologia-spaziatura restano riservate esclusivamente al test e non vengono mai incontrate durante l'addestramento (Tabella 3.4). Questa separazione serve a isolare due assi di generalizzazione: le griglie 5×5 e 6×6 misurano la tenuta della policy su reti più estese di quella di addestramento, con più intersezioni da coordinare, mentre la spaziatura "200m" applicata alla griglia 4×4 misura la tenuta su una topologia mai vista con incroci più distanziati, priva della calibrazione per l'onda verde descritta sopra.

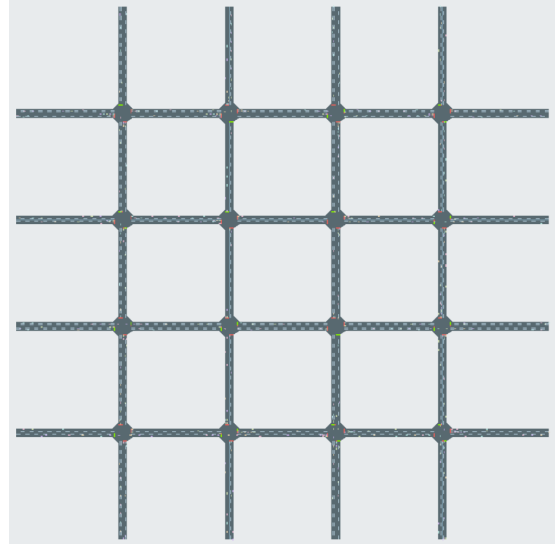
Griglia	Spaziatura	Utilizzo
4×4	100m	Train, Validation, Test
4×4	200m	Test
5×5	100m	Test
6×6	100m	Test

Tabella 3.4: Utilizzo di ciascuna combinazione topologia-spaziatura. Il dettaglio del confine tra training e validazione sulla griglia 4×4 a 100m è in Tabella 3.6.

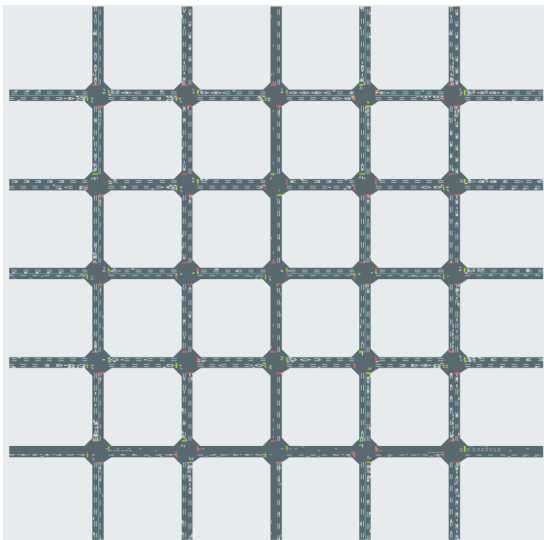
Percorsi veicolari. Il percorso di ciascun veicolo attraverso la rete non è fissato a priori, ma generato casualmente per ciascun punto di ingresso sul perimetro della



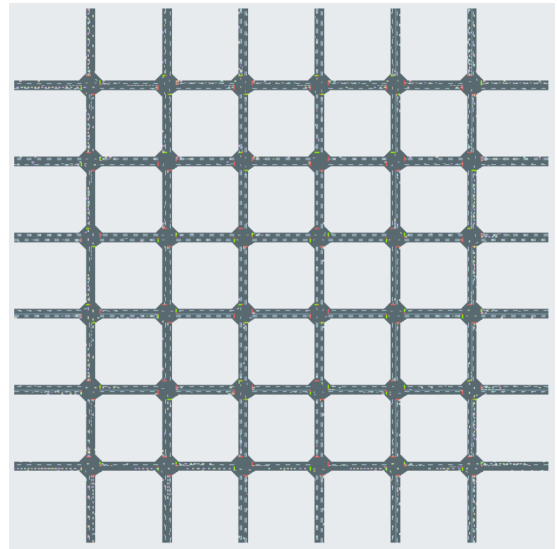
(a) 4×4 , 100m



(b) 4×4 , 200m



(c) 5×5 , 100m



(d) 6×6 , 100m

Figura 3.9: Rendering nel visualizzatore di CITYFLOW delle quattro combinazioni topologia-spaziatura di Tabella 3.4.

griglia: a ogni incrocio attraversato, il percorso prosegue dritto con probabilità 0.60, svolta a destra con probabilità 0.30 o svolta a sinistra con probabilità 0.10, finché non esce dai confini della rete o non ripercorrerebbe un incrocio già attraversato (evitando così percorsi ciclici). La probabilità complessiva di un percorso è il prodotto delle probabilità delle singole scelte che lo compongono; il numero di veicoli assegnato a ciascun percorso è proporzionale al suo peso relativo sul totale dei percorsi generati da quel punto di ingresso, calibrato per raggiungere il volume di traffico target di ciascuna fascia (Tabella 3.5).

Profili di traffico. Per ciascuna topologia sono stati generati tre profili di traffico: un flusso a intensità costante (*flat*), un profilo a gradino (*peak*, diviso in tre fasi temporali con un raddoppio del carico nel terzo centrale) e un profilo dinamico (*complete*) che simula il traffico di una giornata lavorativa (dettagliato nel seguito), impiegato esclusivamente in fase di addestramento. Ogni episodio ha una durata di $T_{\max} = 1800$ s simulati, corrispondenti a $1800 \text{ s} / \Delta t_{\text{step}} = 120$ cicli decisionali per agente.

I volumi dei veicoli immessi nella rete per ciascuna configurazione topologica sono stati determinati tramite una calibrazione empirica, così da garantire un livello di saturazione comparabile tra reti di dimensioni diverse e quindi una difficoltà del task di controllo invariata rispetto la scala topologica. In particolare la densità di ogni topologia è stata bilanciata affinché il *trip completion* ottenuto dalla baseline MAX-PRESSURE risultasse omogeneo (90%–94%).

Profilo di addestramento *complete*. A differenza degli scenari di validazione e test, caratterizzati da densità costanti o a singolo picco, il profilo di addestramento sintetizza l'evoluzione della domanda veicolare di un'intera giornata solare (24 ore) all'interno di un singolo episodio di 1800 s. Come dettagliato nella Tabella 3.5, il flusso è diviso in cinque fasce a intensità variabile: due fasce a basso carico (assimilabili agli orari notturni), due picchi di massima domanda (flussi di traffico di inizio e fine turno lavorativo) e un intervallo di raccordo centrale a regime moderatamente sostenuto. Questa marcata variazione di domanda veicolare all'interno di un singolo episodio è funzionale poiché costringe il modello ad adattare dinamicamente il piano di controllo, senza convergere verso una policy specializzata soltanto su un unico flusso di traffico.

Stocasticità e sbilanciamento direzionale. Per mitigare il rischio di *overfitting* e garantire la robustezza del modello, la generazione del profilo di addestramento integra due accorgimenti.

Fascia	Intervallo simulato	Step di decisione	Veicoli/secondo
Flat basso	0-525s	0-35	1.36
Peak basso	525-675s	35-45	3.67
Flat alto	675-1200s	45-80	2.93
Peak alto	1200-1425s	80-95	4.78
Flat basso	1425-1800s	95-120	1.27

Tabella 3.5: Le cinque fasce del profilo di addestramento *complete* (totale: 4.728 veicoli per `train1`; `train2` e `train3` seguono la stessa struttura con lievi variazioni tra seed, per un totale rispettivamente di 4.723 e 4.620 veicoli).

Innanzitutto, considerata la natura deterministica di CITYFLOW, addestrare un modello su una singola configurazione esporrebbe l'agente all'apprendimento mnemonico di un'esatta sequenza di veicoli. Per prevenire questa eventualità, si generano con diversi *seed* tre profili di traffico differenti, in modo da perturbare la distribuzione dei percorsi e gli istanti di immissione, pur mantenendo la medesima struttura generale descritta in Tabella 3.5; tali varianti vengono quindi alternate ciclicamente durante l'addestramento. Anche le configurazioni di validazione e test, su cui i modelli vengono valutati metodicamente, vengono generate con diversi *seed* (5 per configurazione), così da ottenere statistiche più robuste e informative sui risultati conseguiti.

In secondo luogo, per mettere alla prova la capacità del modello di adattarsi a situazioni locali di traffico differenti, è stata indotta un'asimmetria direzionale, designando una riga orizzontale della griglia come arteria principale (Est-Ovest), con un fattore di potenziamento di 3 che ne aumenta il peso nella distribuzione dei percorsi generati (senza alterare il volume totale di veicoli della configurazione). Tale arteria viene posta su una riga diversa per ogni variante di addestramento, al fine di scongiurare l'ipotesi che le policy sviluppate siano dipendenti dalla posizione dell'incrocio, mentre viene mantenuta fissa sulla stessa riga nelle configurazioni di validazione e di test, in modo da garantire la comparabilità dei risultati.

Training, validation e test. La Tabella 3.6 raccoglie in un unico prospetto tutte le combinazioni di griglia, spaziatura e profilo di traffico introdotte in questo lavoro, con il relativo utilizzo nella pipeline sperimentale; le varianti seed di validazione e test sono omesse, poiché non introducono combinazioni distinte da quelle elencate. Le tre configurazioni di training compaiono anche nella batteria di test, per un confronto diretto con la prassi, diffusa in letteratura, di valutare un modello sulla stessa rete su cui è stato addestrato (Capitolo 4). In tutto possiamo contare 11 configurazioni: 3 di training, 1 di validation e 7 di test.

Griglia	Spaziatura	Profilo di traffico	Utilizzo	Configurazione
4×4	100m	Complete (arteria riga 0)	Train, Test	config_4x4_100m_train1
4×4	100m	Complete (arteria riga 1)	Train, Test	config_4x4_100m_train2
4×4	100m	Complete (arteria riga 2)	Train, Test	config_4x4_100m_train3
4×4	100m	Flat	Validation	config_4x4_100m_6k_flat
4×4	100m	Peak	Test	config_4x4_100m_6k_peak
4×4	200m	Flat	Test	config_4x4_200m_6k_flat
4×4	200m	Peak	Test	config_4x4_200m_6k_peak
5×5	100m	Flat	Test	config_5x5_100m_9.4k_flat
5×5	100m	Peak	Test	config_5x5_100m_9.4k_peak
6×6	100m	Flat	Test	config_6x6_100m_11.5k_flat
6×6	100m	Peak	Test	config_6x6_100m_11.5k_peak

Tabella 3.6: Tutte le combinazioni di griglia, spaziatura e profilo di traffico impiegate in questo lavoro, a meno delle varianti seed di validazione e test, con il relativo utilizzo e il nome del file di configurazione corrispondente.

3.5.4 Assunzioni e limiti del modello di simulazione

Si esplicitano di seguito le semplificazioni strutturali adottate in fase di simulazione, finalizzate a ridurre la complessità computazionale e a rendere il problema trattabile:

- **Fasi transitorie:** Come precedentemente illustrato, l'intervallo semaforico di giallo è assimilato cinematicamente a un prolungamento del verde. La prevenzione dei conflitti è affidata unicamente alla fase di "tutto-rosso".
- **Omogeneità topologica:** L'analisi è confinata a topologie a griglia regolare, con incroci ortogonali e segmenti stradali di lunghezza uniforme. Tale astrazione permette di isolare l'impatto della scala dimensionale e della spaziatura sulle prestazioni di generalizzazione, rimuovendo le distorsioni causate dalle irregolarità geometriche tipiche delle reti urbane reali.
- **Flusso unimodale e omogeneo:** La simulazione modella esclusivamente il traffico veicolare privato. Non sono previsti pedoni, ciclisti, trasporti pubblici o mezzi di soccorso. Inoltre, il flusso è perfettamente omogeneo: ogni veicolo condivide i medesimi parametri cinematici (dimensione, accelerazione massima, velocità massima).
- **Limiti di velocità uniformi:** Il limite di velocità $v_{\max} = 11.1$ m/s è una costante globale, identica per ogni corsia della rete e per ogni veicolo generato, dunque sono assenti gerarchie stradali diverse.
- **Omogeneità degli incroci:** Ogni intersezione è vincolata a una geometria a quattro vie provvista di 8 fasi semaforiche, escludendo di fatto incroci con un numero diverso di vie.

- **Assenza di *lane-changing*:** Le corsie in approccio sono dedicate in modo esclusivo ai singoli movimenti (svolta a sinistra, dritto, svolta a destra). I veicoli vengono generati e si immettono automaticamente nella corsia compatibile con il loro percorso e il cambio di corsia (*lane-changing*) è una dinamica che non è stata implementata.
- **Viabilità bidirezionale:** L'infrastruttura di rete è composta esclusivamente da strade a doppio senso di marcia. La modellazione di strade a senso unico, per quanto frequente nei contesti urbani, non rientra nell'insieme delle configurazioni generate.